

1) Functions with Arguments

1. Write a function `is_anagram(s1, s2)` that checks whether two strings are anagrams.

```
In [2]: def is_anagram(s1, s2):  
        s1 = s1.lower().replace(" ", "")  
        s2 = s2.lower().replace(" ", "")  
  
        if sorted(s1) == sorted(s2):  
            print("Anagram")  
        else:  
            print("Not Anagram")  
  
        is_anagram("listen", "silent")
```

Anagram

2. Write a function that takes a number and prints its square and cube.

```
In [5]: def f1(num):  
  
        print("square:", num ** 2)  
        print("cube: ", num **3)  
  
        f1(10)
```

square: 100
cube: 1000

3. Write a function `intersection(list1, list2)` that returns common elements between two lists without duplicates.

```
In [14]: def intersection(list1, list2):  
        common = []  
  
        for i in list1:  
            if i in list2 and i not in common:  
                common = common + [i]  
  
        return common  
  
        result = intersection([7,6,8,4,5,3], [5,4,3,2,1,9])  
        print(result)
```

[4, 5, 3]

4. Write a function that takes a string and returns the number of vowels in it.

```
In [21]: def count_vowels(text):  
  
        count = 0  
  
        for i in text:  
            if i in "aeiou":
```

```

        count=count + 1

    return count

result =count_vowels("i am learning python")
print(result)

```

6

5. Write a function that takes a list of integers and a target value. Return the indices of two numbers whose sum equals the target.

```

In [22]: def find_indices(numbers, target):
        for i in range(len(numbers)):
            for j in range(i + 1, len(numbers)):
                if numbers[i] + numbers[j] == target:
                    return i, j

result = find_indices([2, 7, 11, 15], 9)
print(result)

```

(0, 1)

2) Default Arguments

1. Write a function generate_password(length=8) that generates a random secure password of given length.

```

In [24]: import random
import string

def generate_password(length=8):
    characters = string.ascii_letters + string.digits + string.punctuation

    password = ""

    for i in range(length):
        password += random.choice(characters)

    return password

print(generate_password())
print(generate_password(12))

```

rX6t61j)
dv\$&4Mkc<g_6

2. Write a function that calculates simple interest with default rate = 5%.

```

In [25]: def simple_interest(principal, time, rate=5):
        interest = (principal * rate * time) / 100
        return interest

```

```
print(simple_interest(20000, 2))
print(simple_interest(20000, 2, 8))
```

2000.0
3200.0

3. Write a function that prints a multiplication table with default number = 5.

```
In [35]: def multi(number=5):
          for i in range(1,11):
              print(f"{number} * {i} = {number * i}")

          multi()
```

5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
5 * 6 = 30
5 * 7 = 35
5 * 8 = 40
5 * 9 = 45
5 * 10 = 50

4. Write a function that calculates the area of a rectangle where the default width is 10.

```
In [36]: def calculate(length,width=10):
          area=length*width
          print(area)

          calculate(8, )
```

80

5. Write a function validate_password (password, min_length=8) that checks password rules (uppercase, lowercase, number, special character).

```
In [4]: def validate_password(password, min_length=8):
          upper = lower = number = special = 0

          for ch in password:
              if ch.isupper():
                  upper += 1
              elif ch.islower():
                  lower += 1
              elif ch.isdigit():
                  number += 1
              else:
                  special += 1

          if len(password) >= min_length and upper > 0 and lower > 0 and number > 0 and special > 0:
              print("Valid Password")
          else:
              print("Invalid Password")
```

```
validate_password("Python@123")  
validate_password("python123")
```

Valid Password

Invalid Password

3) Recursion (Factorial-type Problems)

1. Write a recursive function to calculate the factorial of a number.

```
In [5]: def factorial(n):  
        if n == 1:  
            return 1  
        return n * factorial(n - 1)  
  
print(factorial(5))
```

120

2. Write a recursive function to reverse a string.

```
In [6]: def reverse_string(s):  
        if s == "":  
            return ""  
        return reverse_string(s[1:]) + s[0]  
  
print(reverse_string("Python"))
```

nohtyP

3. Write a recursive function to find the nth Fibonacci number.

```
In [7]: def fibonacci(n):  
        if n == 0:  
            return 0  
        elif n == 1:  
            return 1  
        return fibonacci(n - 1) + fibonacci(n - 2)  
  
print(fibonacci(6))
```

8

4. Write a recursive function to convert a decimal number to binary

```
In [8]: def decimal_to_binary(n):  
        if n == 0:  
            return 0  
        if n == 1:  
            return 1  
        return str(decimal_to_binary(n // 2)) + str(n % 2)  
  
print(decimal_to_binary(10))
```

1010

5. Write a recursive function to check whether a string is palindrome.

```
In [9]: def palindrome(s):
        if len(s) <= 1:
            return True
        if s[0] != s[-1]:
            return False
        return palindrome(s[1:-1])

word = "madam"

if palindrome(word):
    print("Palindrome")
else:
    print("Not Palindrome")
```

Palindrome

4) *args and **kwargs

****1.** Write a function that accepts any number of arguments using *args and returns their sum.*******

```
In [10]: def add(*args):
        total = 0
        for i in args:
            total += i
        return total

print(add(10, 20, 30, 40))
```

100

****2.** Write a function using *args that returns all prime numbers among the arguments.*******

```
In [11]: def prime_numbers(*args):
        for n in args:
            if n > 1:
                count = 0
                for i in range(1, n + 1):
                    if n % i == 0:
                        count += 1
                if count == 2:
                    print(n)

prime_numbers(2, 4, 5, 8, 11, 15, 17)
```

2
5
11
17

3. *Write a function that accepts **kwargs and prints each key-value pair.*******

```
In [12]: def details(**kwargs):
        for key, value in kwargs.items():
            print(key, ":", value)
```

```
details(Name="Ashvini", Age=20, Course="Python")
```

Name : Ashvini
Age : 20
Course : Python

In []: 4. Write a function using `*args` and `**kwargs` to calculate final student grades f additional details.

```
In [13]: def student_grade(*args, **kwargs):
    total = 0

    for mark in args:
        total += mark

    average = total / len(args)

    print("Name :", kwargs["name"])
    print("Course :", kwargs["course"])
    print("Average :", average)

    if average >= 90:
        print("Grade : A")
    elif average >= 75:
        print("Grade : B")
    elif average >= 50:
        print("Grade : C")
    else:
        print("Grade : Fail")

    student_grade(80, 90, 70, name="Ashvini", course="Python")
```

Name : Ashvini
Course : Python
Average : 80.0
Grade : B

*5. Write a function that prints student details using `**kwargs` (name, age, course).***

```
In [14]: def student_details(**kwargs):
    for key, value in kwargs.items():
        print(key, ":", value)

    student_details(name="Ashvini", age=20, course="Python")
```

name : Ashvini
age : 20
course : Python

5) Return Statement

1. Write a function that returns the square of a number.

```
In [15]: def square(n):
    return n * n

print(square(5))
```

25

2. Write a function that returns the longest word in a sentence.

```
In [16]: def longest_word(sentence):
          words = sentence.split()
          longest = words[0]

          for word in words:
              if len(word) > len(longest):
                  longest = word

          return longest

print(longest_word("I am learning Python Programming"))
```

Programming

3. Write a function that returns True if a number is an Armstrong number, otherwise False.

```
In [17]: def armstrong(n):
          temp = n
          total = 0

          while temp > 0:
              digit = temp % 10
              total = total + digit ** 3
              temp = temp // 10

          return total == n

print(armstrong(153))
print(armstrong(123))
```

True

False

4. Write a function that returns a list of numbers divisible by both 3 and 5 between two numbers.

```
In [18]: def divisible(start, end):
          result = []

          for i in range(start, end + 1):
              if i % 3 == 0 and i % 5 == 0:
                  result.append(i)

          return result

print(divisible(1, 100))
```

[15, 30, 45, 60, 75, 90]

5. Write a function that returns the number of uppercase, lowercase, and digits in a string.

```
In [20]: def count_characters(text):
          upper = 0
```

```

lower = 0
digit = 0

for ch in text:
    if ch.isupper():
        upper += 1
    elif ch.islower():
        lower += 1
    elif ch.isdigit():
        digit += 1

return upper, lower, digit

u, l, d = count_characters("Python123ABC")

print("Uppercase:", u)
print("Lowercase:", l)
print("Digits:", d)

```

Uppercase: 4
 Lowercase: 5
 Digits: 3

6) Global vs Local Variables

1. Write a program demonstrating the difference between global and local variables.

```

In [21]: x = 10      # Global variable

def demo():
    y = 20 # Local variable
    print("Inside function")
    print("Global:", x)
    print("Local:", y)

demo()

print("Outside function")
print("Global:", x)

```

Inside function
 Global: 10
 Local: 20
 Outside function
 Global: 10

2. Write a function that modifies a global variable using the global keyword.

```

In [22]: count = 10

def change():
    global count
    count = 20

change()

```



```
print(count)
```

20

3. Write a function where a local variable has the same name as a global variable and observe the output.

```
In [23]: name = "Ashvini"

def student():
    name = "Python"
    print("Inside:", name)

student()

print("Outside:", name)
```

Inside: Python

Outside: Ashvini

4. Write a program that counts how many times a function is called using a global counter.

```
In [24]: count = 0

def display():
    global count
    count += 1
    print("Function called", count, "times")

display()
display()
display()
```

Function called 1 times

Function called 2 times

Function called 3 times

5. Write a program that shows the scope of variables inside and outside functions.

```
In [25]: a = 100

def test():
    b = 200
    print("Inside function")
    print("a =", a)
    print("b =", b)

test()

print("Outside function")
print("a =", a)
# print(b) # Error: b is local to the function
```

Inside function

a = 100

b = 200

Outside function

a = 100

